MACHINE LEARNING • MODULE 05

Support Vector Machines

Maximum Margin Classifiers

■ SVM

■ LEARNING OUTCOME

By the end of this module you will understand the maximum margin hyperplane concept, implement linear and kernel SVMs, tune C and gamma, use SVR for regression, and know when SVMs beat other algorithms.

■ Skills You Will Gain

- Explain the maximum margin hyperplane and support vectors
- Implement linear SVM for classification
- Apply RBF and polynomial kernels for non-linear data
- Use SVR for regression problems
- Tune C (regularisation) and gamma (kernel width)
- Know when SVMs outperform tree-based models

■ **SVMs dominated ML benchmarks for a decade. They still excel for text classification, bioinformatics, image recognition, and small high-dimensional datasets.**

SECTION 1

SVM Intuition — Maximum Margin

Core Concept

SVM finds the hyperplane that separates classes with the MAXIMUM MARGIN. Support Vectors: the training points closest to the boundary. ONLY these points define the model. Wider margin = better generalisation = less overfitting. C parameter controls the trade-off: Small C: wider margin, more misclassification allowed (soft margin) Large C: narrow margin, fewer misclassifications (hard margin, more overfit)

Term	Meaning
Hyperplane	Decision boundary: w.x + b = 0 — separates classes
Margin	Distance between hyperplane and nearest support vectors — maximise this
Support Vectors	Only data points that define the margin — removing others doesn't change model
Soft Margin	Allow some misclassifications controlled by C — used when data not linearly separable
Kernel	Function that maps data to higher dimensions implicitly — enables non-linear boundaries
Dual Problem	SVM solved using Lagrange multipliers — efficient for high-dimensional data

SECTION 2

Linear SVM

```
from sklearn.svm import SVC, SVR from sklearn.preprocessing import StandardScaler from sklearn.pipeline import Pipeline # CRITICAL: Always scale before SVM — it uses distances! svm_pipe = Pipeline([ ('scaler', StandardScaler()), ('svm', SVC(kernel='linear', C=1.0, probability=True)) ]) svm_pipe.fit(X_train, y_train) y_pred = svm_pipe.predict(X_test) y_proba = svm_pipe.predict_proba(X_test)[:, 1] # Inspect support vectors print(f'Number of support vectors: {svm_pipe["svm"].n_support_}')
```

SECTION 3

Kernel Trick for Non-Linear Data

	Formula	Best For
	x . x'	Text, high-dimensional, linearly separable data
	exp(-g||x-x'||^2)	Default for tabular — most non-linear problems
	(g*x.x' + r)^d	Image recognition, specific geometric shapes
	tanh(g*x.x' + r)	Rarely used — similar to one neural network layer

```
# RBF kernel — best general-purpose non-linear SVM svm_rbf = Pipeline([ ('scaler', StandardScaler()), ('svm', SVC(kernel='rbf', C=10, gamma='scale', probability=True)) ]) # Tune C and gamma with GridSearch from sklearn.model_selection import GridSearchCV param_grid = { 'svm__C': [0.1, 1, 10, 100], 'svm__gamma': ['scale', 'auto', 0.01, 0.1] } grid = GridSearchCV(svm_rbf, param_grid, cv=5, scoring='f1', n_jobs=-1) grid.fit(X_train, y_train) print(f'Best params:
```

```
{grid.best_params_}') print(f'Best F1: {grid.best_score_:.4f}')
```

SECTION 4

SVM for Regression (SVR)

SVR — Support Vector Regression

SVR fits a tube of width 2*epsilon around the data. Points INSIDE the tube have zero loss. Points outside the tube are penalised proportional to their distance from the tube. This makes SVR robust to outliers — unlike linear regression which penalises all residuals equally.

```
from sklearn.svm import SVR svr = Pipeline([ ('scaler', StandardScaler()), ('svr',
SVR(kernel='rbf', C=100, epsilon=0.1, gamma='scale')) ]) svr.fit(X_train, y_train) y_pred =
svr.predict(X_test) from sklearn.metrics import mean_absolute_error, r2_score print(f'MAE:
{mean_absolute_error(y_test, y_pred):.3f}') print(f'R2: {r2_score(y_test, y_pred):.4f}')
```

■ PRACTICE Q & A

Q1. What are support vectors and why do only they define the SVM model?

A: *Training points closest to the decision boundary. The hyperplane is determined by maximising margin with respect to these points. Removing non-support vectors doesn't change the model.*

Q2. What does increasing C do to an SVM?

A: *Higher C = stricter about misclassifications = narrower margin = more complex boundary = more risk of overfitting. Lower C = wider margin = simpler model = more robust.*

Q3. Why MUST you scale features before training an SVM?

A: *SVM uses distances. Features with large magnitude dominate the distance calculation, making the model biased. StandardScaler ensures equal contribution from all features.*

Q4. What is the kernel trick?

A: *Maps data to higher-dimensional space where it becomes linearly separable, but does so implicitly without computing the actual high-dim coordinates. Computationally efficient and powerful.*

Q5. When would you choose SVM over Random Forest?

A: *Text classification (sparse high-dim features), small datasets with many features, bioinformatics data. Random Forest is usually better for large tabular datasets with many samples.*

■ SVM Cheat Sheet	
`SVC(kernel='linear', C=1)`	Linear SVM classifier
`SVC(kernel='rbf', C=10)`	RBF SVM — non-linear
`SVR(kernel='rbf', C=1)`	SVM for regression
`probability=True`	Enable predict_proba()
`C parameter`	Low C = wider margin, simpler
`gamma='scale'`	Auto-set gamma = 1/(n_features*X.var())
`epsilon in SVR`	Width of tube with no loss
`StandardScaler()`	ALWAYS scale before SVM
`Pipeline([scaler, svm])`	Chain preprocessing + model
`GridSearchCV(pipe, params)`	Tune C and gamma together
`kernel='linear'`	For text, sparse, high-dim
`kernel='rbf'`	Default for most tabular data